BÁO CÁO CHIÊM NGHIỆM VÀ TỔNG KẾT — TRANSFORMER ARCHITECTURE

Tài Liệu Phân Tích & Thực Nghiệm Khoa Học

November 4, 2026

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng
Học kỳ: Học kỳ I - 2026
Chủ đề: From Attention to Transformer
Sinh viên thực hiện: Nguyễn Văn Thành
Hạn nộp: 23:59 - 04/11/2026
—

1. Kiểm Chứng Thực Nghiệm 6 Dự Đoán Ban Đầu

Theo đúng nguyên tắc khoa học tại Mục 28 của đề bài `W7.pdf`, bảng dưới đây đối chiếu chi tiết giữa giả thuyết lý thuyết ban đầu và số liệu đo đạc thực tế từ các thí nghiệm:

STT	Câu hỏi giả thuyết	Dự đoán ban đầu	Kết quả thực nghiệm	Đánh giá	Diễn giải nguyên nhân khoa học
1	Transformer xử lý song song hơn RNN?	**CÓ**	Khẳng định qua phép nhân ma trận toàn cục.	**ĐÚNG**	Toàn bộ chuỗi được chiếu và tính QK^T đồng thời, không bị nghẽn tuần tự $h_t = f(h_{t-1}, x_t)$ như LSTM.
2	Bỏ Positional Encoding có biết thứ tự token?	**KHÔNG**	Permutation test cho biểu diễn đồng nhất.	**ĐÚNG**	Self-Attention có tính hoán vị đồng biến; khi kết hợp với Mean Pooling toàn cục, thứ tự bị triệt tiêu hoàn toàn.
3	Phân biệt được hai câu cùng token khác thứ tự nếu không có PE?	**KHÔNG**	Khoảng cách L2 sau pooling xấp xỉ 0.	**ĐÚNG**	Hai câu S_1 (”dog bites man”) và S_2 (”man bites dog”) có cùng đa tập hợp vector đầu vào nên cho cùng một xác suất dự đoán.
4	Các attention heads có nhất thiết học cùng một pattern?	**KHÔNG**	4 heads học 4 kiểu phân bổ khác nhau.	**ĐÚNG**	Head 1 tập trung cú pháp cục bộ, Head 2 tập trung từ cảm xúc, Head 3 chú ý liên từ, Head 4 phân tán làm mịn.
5	Tăng số heads có chắc chắn tăng accuracy?	**KHÔNG**	Accuracy dao động: 70.2% (1h), 71.6% (2h), 71.0% (4h), 72.7% (8h).	**ĐÚNG**	Khi d_{model} cố định (128), tăng heads làm giảm d_k (từ 128 xuống 16), giới hạn không gian biểu diễn của tích vô hướng từng head.
6	Transformer luôn nhanh hơn RNN trên mọi độ dài chuỗi?	**KHÔNG**	$T = 32$ tốn 14.1s; $T = 256$ tốn 163.0s (tăng gấp 11.5 lần).	**ĐÚNG**	Ma trận Attention bùng nổ theo bậc hai $O(T^2)$. Ở chuỗi dài, chi phí bộ nhớ và tính toán T^2 lấn át ưu thế song song.

—

2. Bảng So Sánh Toàn Bộ Chuỗi Mô Hình: LAB 04 → LAB 07

Bảng dưới đây tổng hợp sự tiến hóa về mặt kiến trúc và năng lực biểu diễn qua 4 bài thực hành cốt lõi của môn học (theo Mục 27 trong `W7.pdf`):

Thuộc tính (Property)	**TF-IDF (LAB 04)**	**Avg Word2Vec (LAB 04)**	**LSTM (LAB 05)**	**Self-Attention (LAB 06)**	**Transformer (LAB 07)**
Biểu diễn (Representation)	Rời rạc, Thưa (Sparse)	Dày đặc, Tĩnh (Dense, Static)	Dày đặc, Động (Dense, Dynamic)	Dày đặc, Động (Dense, Dynamic)	Dày đặc, Đa không gian con
Thứ tự từ (Word Order)	Rất hạn chế (N-gram)	Không có (Bag-of-Vectors)	Tự nhiên theo thời gian (t)	Cần Positional Information	Mã hóa tường minh qua PE
Phụ thuộc ngữ cảnh (Context)	Không	Không	Một chiều / Gián tiếp qua h_t	Trực tiếp giữa mọi cặp từ	Trực tiếp + Đa đầu + Phi tuyến FFN
Tương tác trực tiếp token	Không	Không	Hạn chế (Bị nén qua các bước)	Hoàn toàn trực tiếp qua QK^T	Hoàn toàn trực tiếp qua Multi-Head
Tính toán tuần tự (Sequential)	Không	Không	Bắt buộc ($O(T)$ bước)	Không	Không (Hoàn toàn song song)
Khả năng song song hóa	Rất cao	Rất cao	Rất kém	Rất cao	Rất cao
Tương tác khoảng cách xa	Rất kém	Không có	Rất khó (Vanishing Gradient)	Tốt (Đường truyền trực tiếp)	Cực tốt (Attention + Residual Highway)
Chi phí theo độ dài chuỗi (T)	Tuyến tính $O(T)$	Tuyến tính $O(T)$	Tuyến tính $O(T \cdot d^2)$	Bậc hai $O(T^2 \cdot d)$	Bậc hai $O(T^2 \cdot d)$

—

3. Mạch Nối Tiến Hóa Kiến Trúc: Từng Bước Đã Giải Quyết Giới Hạn Nào?

Chuỗi chương trình học từ LAB 01 đến LAB 07 là một lộ trình sư phạm mạch lạc, mỗi bước ra đời nhằm khắc phục trực tiếp các nút thắt kỹ thuật của bước trước đó:

1. **TF-IDF (LAB 01 / LAB 04)**:

- *Ưu điểm*: Đơn giản, tính toán nhanh, vector thưa hiệu quả cho bài toán tra cứu văn bản.
- *Hạn chế*: Không gian chiều cực lớn (Curse of Dimensionality), ma trận thưa, hoàn toàn bỏ qua ngữ nghĩa từ đồng nghĩa (Orthogonal vectors) và không có trật tự từ.

1. **Word2Vec (LAB 03 / LAB 04)**:

- *Giải quyết*: Nén từ vựng vào không gian vector dày đặc (dense embeddings), nắm bắt được tương đồng ngữ nghĩa hình học ($\cos(\mathbf{u}, \mathbf{v})$).
- *Hạn chế*: Biểu diễn tĩnh (static embedding); một từ nhiều nghĩa (polysemy như ”bank”) chỉ có duy nhất một vector cố định; phép cộng trung bình bỏ qua trật tự câu.

1. **RNN / LSTM / GRU (LAB 05)**:

- *Giải quyết*: Xử lý chuỗi theo trật tự thời gian, tạo ra biểu diễn phụ thuộc ngữ cảnh động (dynamic hidden states).
- *Hạn chế*: Điểm nghẽn thông tin (Information Bottleneck) khi phải nén toàn bộ chuỗi dài vào một vector h_T duy nhất; triệt tiêu đạo hàm (vanishing gradient) qua nhiều bước thời gian; không thể tính toán song song trên GPU.

1. **Self-Attention (LAB 06)**:

- *Giải quyết*: Phá bỏ điểm nghẽn nén chuỗi, cho phép mọi token kết nối trực tiếp với nhau với khoảng cách đường truyền gradient bằng $O(1)$; tính toán hoàn toàn song song.
- *Hạn chế*: Chỉ có một head chú ý duy nhất (không học được nhiều loại quan hệ đồng thời); thiếu tính chất phi tuyến kênh (channel mixing); không có thông tin vị trí từ nếu đứng độc lập.

1. **Transformer Architecture (LAB 07)**:

- *Giải quyết*: Kết hợp hoàn chỉnh Positional Encoding + Multi-Head Attention + Residual Connections + Layer Normalization + Position-wise FFN.
- *Giới hạn còn lại (Remaining Limitations)*:
- Độ phức tạp bộ nhớ và tính toán bậc hai $O(T^2)$ theo độ dài chuỗi, gây nghẽn nghiêm trọng khi văn bản dài hàng nghìn token.
- Mô hình có số lượng tham số lớn (> 1.4 triệu tham số cho mô hình nhỏ), nếu huấn luyện từ đầu (training from scratch) trên tập dữ liệu khiêm tốn sẽ dễ bị quá khớp (overfitting) và chưa tận dụng được kho tri thức ngôn ngữ khổng lồ.

—

4. Trả Lời Bộ Câu Hỏi Vấn Đáp Nhanh (Individual Learning Check)

Dưới đây là câu trả lời chuẩn xác cho 9 câu hỏi kiểm tra ngẫu nhiên tại Mục 31 của `W7.pdf`:

Câu 1: Tại sao Transformer cần Positional Encoding?

Trả lời: Phép tính tích vô hướng Self-Attention và phép cộng pooling có tính chất bất biến hoán vị (permutation equivariance / invariance). Nếu không có Positional Encoding, mô hình không thể phân biệt được câu chủ động và câu bị động (”dog bites man” và ”man bites dog”). Positional Encoding cung cấp thông tin toạ độ thứ tự duy nhất cho từng vị trí token.

Câu 2: Multi-Head Attention khác một Attention Head ở đâu?

Trả lời: Một attention head chỉ chiếu thông tin vào một không gian con duy nhất d_k, do đó chỉ học được một kiểu tương tác (ví dụ hoặc chỉ chú ý từ ngữ pháp, hoặc chỉ chú ý từ cảm xúc). Multi-Head Attention chia nhỏ vector thành h không gian con độc lập chạy song song, cho phép mô hình đồng thời nắm bắt nhiều loại quan hệ cú pháp và ngữ nghĩa khác nhau tại các vị trí khác nhau trong chuỗi.

Câu 3: Nếu $d_{model} = 128$ và có 8 heads thì d_k bằng bao nhiêu?

Trả lời: Theo công thức phân rã không gian chuẩn:

$$d_k = \frac{d_{model}}{n_{heads}} = \frac{128}{8} = 16$$

Mỗi head có số chiều là 16.

Câu 4: Tại sao Attention Matrix có kích thước $T \times T$?

Trả lời: Vì Attention Matrix phản ánh mức độ liên kết tương tác giữa mỗi token trong số T token đầu vào với tất cả T token khác trong chuỗi (tính cả chính nó). Phần tử tại hàng i, cột j biểu thị trọng số chú ý mà token thứ i dành cho token thứ j.

Câu 5: Residual Connection có vai trò gì trong Transformer Block?

Trả lời: Residual connection ($x + F(x)$) tạo ra một "đường cao tốc gradient" (gradient highway), giúp tín hiệu lan truyền ngược truyền thẳng từ các tầng sâu về các tầng nông mà không bị suy giảm theo cấp số nhân. Nó ngăn chặn hiện tượng mất mát thông tin biểu diễn gốc và giúp việc tối ưu hóa mạng nơ-ron sâu trở nên ổn định và dễ hội tụ.

Câu 6: Feed-Forward Network làm gì trong Transformer Block?

Trả lời: Multi-Head Attention chủ yếu thực hiện việc pha trộn thông tin ngang giữa các vị trí trong chuỗi (token mixing). Tầng Position-wise FFN thực hiện việc biến đổi phi tuyến dọc theo chiều đặc trưng tại từng vị trí độc lập (channel mixing), giúp mô hình học các tổ hợp đặc trưng trừu tượng và nâng cao năng lực xấp xỉ hàm.

Câu 7: Tại sao Transformer có thể xử lý các token song song hơn RNN?

Trả lời: Transformer không có vòng lặp thời gian $h_t = f(h_{t-1}, x_t)$. Toàn bộ các phép biến đổi Query, Key, Value và tích vô hướng ma trận QK^T đều được thực thi dưới dạng phép nhân ma trận đồng thời trên toàn bộ batch và sequence thông qua các toán tử tensor được tối ưu hóa cao độ trên GPU/CPU.

Câu 8: Nếu sequence dài gấp đôi, Attention Matrix thay đổi như thế nào?

Trả lời: Số lượng phần tử trong Attention Matrix tăng theo hàm bậc hai:

$$(2T) \times (2T) = 4T^2$$

Tức là kích thước ma trận và chi phí bộ nhớ lưu trữ tăng gấp **4 lần** (tăng 400%).

Câu 9: Vì sao không thể kết luận rằng attention weight cao đồng nghĩa token đó là "lý do" mô hình đưa ra dự đoán?

Trả lời: Bởi vì Attention chỉ là trọng số tổ hợp tuyến tính ở tầng trung gian ($z = \sum A_{ij} v_j$). Sau đó, vector này còn phải đi qua kết nối tắt Residual, tầng LayerNorm và hai tầng phi tuyến FFN trước khi ra quyết định. Trọng số Attention phản ánh luồng tương tác dữ liệu nội tại của mô hình (model behavior), chứ không phải là mối quan hệ nhân quả (causal explanation) dẫn đến quyết định phân loại cuối cùng.

—

5. Mạch Nối Tự Nhiên Sang LAB 08: Pretrained Transformer và Kiến Trúc BERT

Ở LAB 07, chúng ta đã tự tay xây dựng hoàn chỉnh một Transformer Encoder block từ các thành phần nguyên thủy. Tuy nhiên, mô hình ở LAB 07 được **huấn luyện từ đầu (trained from scratch)** trên tập dữ liệu IMDb khiêm tốn ($10,000$ mẫu).

Điều này dẫn đến câu hỏi nghiên cứu mang tính bước ngoặt:

> *Nếu Transformer có năng lực học biểu diễn mạnh mẽ đến vậy, tại sao chúng ta không huấn luyện trước một Transformer Encoder khổng lồ trên hàng tỷ từ vựng (Wikipedia, BookCorpus) để nó nắm vững toàn bộ ngữ pháp, tri thức và ngữ nghĩa của nhân loại, sau đó chỉ việc tinh chỉnh (fine-tune) nhẹ nhàng cho từng bài toán cụ thể?*

Đó chính là tư tưởng khai sinh ra **BERT (Bidirectional Encoder Representations from Transformers)** và kỷ nguyên mô hình ngôn ngữ lớn (LLMs). Khi bước sang **LAB 08**, sinh viên sẽ không còn bỡ ngỡ với các khái niệm `[CLS]`, `[SEP]`, Masked Attention hay Token-Type Embeddings, bởi vì toàn bộ cấu trúc nền tảng (Multi-Head Attention, Residual, LayerNorm, FFN, Positional Information) đều đã được làm chủ tường minh trong bài thực hành LAB 07 này.